React Patterns Cheat Sheet

React + TypeScript patterns — quick reference

---

1. useState — Declaring State

stores a value in a component. setter triggers re-render.

Basic string state

App.tsx — const [search, setSearch]

```
const [search, setSearch] = useState("")
// [0] = current value    [1] = function to update it

// To update:
setSearch("AAPL")  // React re-renders with new value
```

> always go through the setter. assigning directly (search = 'AAPL') doesn't trigger a re-render.

Array state

App.tsx — const [portfolioValues, setPortfolioValues]

```
const [portfolioValues, setPortfolioValues] = useState<string[]>([]);

// ❌ WRONG — mutates in place, React won't re-render:
portfolioValues.push("AAPL");  // Never do this!

// ✅ CORRECT — always pass a brand new array:
setPortfolioValues([...portfolioValues, "AAPL"]);
```

> React compares by reference. .push() keeps the same reference → no re-render. [...arr] makes a new array → new reference → re-render.

Error / status state

App.tsx — const [serverError, setServerError]

```
const [serverError, setServerError] = useState<string>("")

// On error:
setServerError("Network error!")

// Show in JSX only when there is an error:
{serverError && <h1>{serverError}</h1>}
```

> empty string is falsy → nothing renders. once it has content, the h1 shows.

---

2. Add an Item to an Array

Spread operator pattern

App.tsx — onPortfolioCreate

```
const onPortfolioCreate = (e: any) => {
  e.preventDefault();  // stop page reload first

  // Step 1: read the value from the form's first input
  const newValue = e.target[0].value;

  // Step 2: create new array = all old items + new item at end
  const updated = [...portfolioValues, newValue];

  // Step 3: update state
  setPortfolioValues(updated);
};
```

> ...portfolioValues copies the existing items, then newValue is appended.

Add only if it doesn't already exist (no duplicates)

App.tsx — onPortfolioCreate (exists check before add)

```
const onPortfolioCreate = (e: any) => {
  e.preventDefault();
  const newValue = e.target[0].value;

  // .find() returns the item if found, or undefined if not
  const exists = portfolioValues.find((val) => val === newValue);

  if (exists) return;  // already in portfolio, stop here

  setPortfolioValues([...portfolioValues, newValue]);
};
```

> .find() returns the item (truthy) or undefined (falsy). truthy → early return → setState never runs.

---

3. Delete an Item from an Array

The .filter() pattern

App.tsx — onPortfolioDelete

```
const onPortfolioDelete = (e: any) => {
  e.preventDefault();
  const toRemove = e.target[0].value;

  // .filter() keeps items where the condition is TRUE
  // so: keep everything that is NOT the item to remove
  const remaining = portfolioValues.filter((val) => val !== toRemove);
```

```
  setPortfolioValues(remaining);
};
```

Step-by-step mental model of .filter()

```
// Original array:
["AAPL", "TSLA", "GOOG"]

// .filter(val => val !== "TSLA")

//  "AAPL" !== "TSLA"  →  true   →  KEPT
//  "TSLA" !== "TSLA"  →  false  →  DROPPED
//  "GOOG" !== "TSLA"  →  true   →  KEPT

// Result: ["AAPL", "GOOG"]
```

> .filter() always returns a new array — never mutates. fits state updates.

---

4. Render a List with .map()

Turn array items into components

CardList.tsx, ListPortfolio.tsx

```
// .map() runs a function on every item and returns an array of JSX
portfolioValues.map((portfolioValue) => (
  <CardPortfolio
    portfolioValue={portfolioValue}
    onPortfolioDelete={onPortfolioDelete}
  />
))
```

> each item → one component, rendered in order.

Show empty message when array has no items

ListPortfolio.tsx, CardList.tsx — ternary empty-state check

```
{portfolioValues.length > 0 ? (
  portfolioValues.map((val) => <CardPortfolio portfolioValue={val} ... />)
) : (
  <h3>Your portfolio is empty.</h3>
)}
```

> length > 0 = 'any items?'. ternary swaps between list and empty message.

---

5. Handling Events

Controlled input — typing in a search box

Search.tsx + App.tsx — handleSearchChange

```
// State holds the current value of the input
const [search, setSearch] = useState("")

// Called every time the user types a character
const handleSearchChange = (e: ChangeEvent<HTMLInputElement>) => {
  setSearch(e.target.value);
  // e.target.value = whatever is currently typed in the box
};

// In JSX — both value and onChange are required:
<input value={search} onChange={handleSearchChange} />
```

> value= shows state. onChange= writes to state. need both — without onChange, the input looks frozen.

e.preventDefault() — always the first line in form handlers

App.tsx — every onSubmit handler

```
const onSearchSubmit = (e: SyntheticEvent) => {
  e.preventDefault();  // stops the browser from refreshing the page

  // your logic here — state is preserved because page didn't reload
};

<form onSubmit={onSearchSubmit}>...</form>
```

> HTML forms reload the page on submit and wipe React state. preventDefault cancels that.

---

6. Reading Values from a Form

Hidden input trick — pass data through a form without showing it

AddPortfolio.tsx, DeletePortfolio.tsx

```
// JSX — the hidden input carries the symbol, the button triggers the form
<form onSubmit={onPortfolioCreate}>
  <input readOnly={true} hidden={true} value={symbol} />
  <button type="submit">Add</button>
</form>

// Handler — e.target is the form, e.target[0] is its first input
const onPortfolioCreate = (e: any) => {
  e.preventDefault();
  const symbol = e.target[0].value;  // reads the hidden input's value
};
```

> e.target = the form. e.target[0] = first input, [1] = second, etc. hidden inputs still readable in the handler.

7. Async API Calls

The API function with try/catch

api.tsx — searchCompanies

```
export const searchCompanies = async (query: string) => {
  try {
    const data = await axios.get(url);
    return data;  // returns the axios response object
  } catch (error) {
    if (axios.isAxiosError(error)) return error.message;  // returns a string
    return "An unexpected error occurred";
  }
};
```

Using the result — check type before using data

App.tsx — onSearchSubmit

```
const onSearchSubmit = async (e: SyntheticEvent) => {
  e.preventDefault();

  // await pauses here until the API responds
  const result = await searchCompanies(search);

  // If it returned a string, it's an error message
  if (typeof result === "string") {
    setServerError(result);
  } else if (Array.isArray(result.data)) {
    // result.data is the array of company results
    setSearchResult(result.data);
  }
};
```

> api fn returns a string on error, axios response on success. typeof catches the error case, Array.isArray confirms there's data to use.

8. Array Search Methods

.find() — search for an item

App.tsx — onPortfolioCreate (duplicate check)

```
// Returns the item itself if found, or undefined if not
const exists = portfolioValues.find((val) => val === newValue);

// Also works for objects — match by property:
const company = searchResults.find((c) => c.symbol === "AAPL");
```

.includes() — simpler true/false check

```
// Returns true or false directly
const alreadyAdded = portfolioValues.includes("AAPL");

if (!portfolioValues.includes(symbol)) {
  setPortfolioValues([...portfolioValues, symbol]);
}
```

> .includes() for primitives. .find() for objects (match on a property).

.some() — check if any item matches a condition

```
// Returns true if at least one item passes the test
const hasNasdaq = searchResults.some((c) => c.exchange === "NASDAQ");

// Equivalent duplicate check using .some():
if (portfolioValues.some((val) => val === newValue)) return;
```

---

9. Conditional Rendering in JSX

&& — show element only when truthy

App.tsx — serverError display

```
{serverError && <h1>{serverError}</h1>}

// serverError = ""       →  nothing renders (empty string is falsy)
// serverError = "Error"  →  <h1>Error</h1> renders
```

Ternary ? : — show A or B

CardList.tsx, ListPortfolio.tsx — ternary swap

```
{searchResults.length > 0 ? (
  searchResults.map((result) => <Card searchResult={result} ... />)
) : (
  <p>No results!</p>
)}

// Read as: condition ? (show if true) : (show if false)
```

> && = show / hide. ternary = swap A vs B.

---

10. Passing Functions as Props

Define in parent (where state lives), pass down to children

App.tsx → CardList → Card → AddPortfolio

```
// App.tsx — function defined here because state is here
const onPortfolioCreate = (e: any) => { ... };
<CardList onPortfolioCreate={onPortfolioCreate} />

// CardList.tsx — receives it in Props, passes further down
interface Props { onPortfolioCreate: (e: SyntheticEvent) => void; }
const CardList = ({ onPortfolioCreate }: Props) => {
  return <Card onPortfolioCreate={onPortfolioCreate} ... />
};

// AddPortfolio.tsx — the actual button that calls it
<form onSubmit={onPortfolioCreate}>...</form>
```

> state and its setters live together at the top. children just call the function — the update still runs at the top.

---

11. TypeScript Props Interface

Describe what a component expects

every component file

```
// Define the shape at the top of the file
interface Props {
  portfolioValues: string[];           // array of strings
  onPortfolioDelete: (e: SyntheticEvent) => void; // function, returns nothing
  searchResult: CompanySearch;         // custom type from company.ts
  id: string;
}

// Use it in the component:
const Card = ({ id, searchResult, onPortfolioCreate }: Props) => {
  return <div>...</div>;
};
```

> void = function returns nothing. TS errors out on wrong types or missing required props.

---

12. useEffect — Sync with the Outside World

runs after render. for fetches, timers, subscriptions — anything outside React.

Three usage patterns

```
// 1. No dependency array — runs after EVERY render (rare; usually a bug)
useEffect(() => {
  // runs every time
```

```
});

// 2. Empty array [] — runs ONCE on mount only (most common for fetching)
useEffect(() => {
  fetchData();
}, []);

// 3. With dependencies [x] — re-runs whenever x changes
useEffect(() => {
  fetchData(id);
}, [id]);
```

> deps are compared by reference. primitives compare by value. objects/arrays by reference — a fresh object each render counts as changed even if contents are identical.

Async work inside useEffect

effect callback can't be async (must return nothing or a cleanup fn, not a Promise). define an async fn inside and call it.

```
useEffect(() => {
  const init = async () => {
    const result = await fetchData(id);
    setData(result);
  };
  init();
}, [id]);
```

Cleanup function

return a fn to undo the effect — clear timers, remove listeners, cancel subscriptions. runs before next effect + on unmount.

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);  // cleanup
}, []);
```

---

13. React Router — Multi-Page Apps

maps URLs to components, no full reloads. createBrowserRouter holds the tree, RouterProvider mounts it.

Route tree with nested children

```
import { createBrowserRouter, RouterProvider } from "react-router-dom";

export const router = createBrowserRouter([{
  path: "/",
  element: <Layout />,           // parent layout
  children: [
    { path: "",           element: <Home /> },     // /
    { path: "about",      element: <About /> },    // /about
    { path: "item/:id",   element: <Item /> },     // /item/123  (URL param)
```

```
  ],
}]);

// In index.tsx — mount the router
<RouterProvider router={router} />
```

<Outlet /> — where child routes render

parent layout places <Outlet /> where the child route should slot in. anything around it (Navbar etc.) stays mounted across navigations.

```
import { Outlet } from "react-router-dom";

const Layout = () => (
  <>
    <Navbar />
    <Outlet />        // child route renders here
  </>
);
```

<Link> vs <a>

<Link to="/path"> = in-app nav, no reload, state preserved. <a href> = full reload, state wiped.

```
import { Link } from "react-router-dom";

<Link to={"/"}>Home</Link>
<Link to={"about"}>About</Link>      // relative path (joins parent)
```

> pages own state + fetching. components stay presentational, receive via props. keeps components reusable.

---

14. URL Parameters — useParams

route path uses :name, useParams() reads it. for detail pages where the URL identifies the resource.

```
// Route definition: path is "item/:id"

import { useParams } from "react-router-dom";

const ItemPage = () => {
  const { id } = useParams();    // id is string | undefined

  useEffect(() => {
    if (id) fetchItem(id);
  }, [id]);                // re-fetch when URL changes

  return <div>Item {id}</div>;
};
```

> useParams returns string | undefined. guard with if (id) or assert with id! when the route guarantees it. include the param in useEffect deps so navigation refetches.

---

15. Config-Driven Rendering

separate WHAT from HOW. configs array of {label, render}, mapped twice — once for headers, once per row. add a column = push one object.

```
type Row = { name: string; price: number; year: number };

const configs = [
  { label: "Name",  render: (r: Row) => r.name },
  { label: "Price", render: (r: Row) => r.price },
  { label: "Year",  render: (r: Row) => r.year },
];

const headers = configs.map((c) => <th key={c.label}>{c.label}</th>);

const rows = data.map((row) => (
  <tr key={row.name}>
    {configs.map((c) => <td key={c.label}>{c.render(row)}</td>)}
  </tr>
));

<table><thead><tr>{headers}</tr></thead><tbody>{rows}</tbody></table>
```

> every .map() child needs a unique key. use a stable id from the data — array index only if items never reorder/insert/delete.

---

16. children Prop & <Outlet /> Together

children = passed by parent in JSX. Outlet = filled by router based on URL. layouts can use both.

```
type Props = { children: React.ReactNode };

const Dashboard = ({ children }: Props) => (
  <div>
    {children}        // content passed by the parent in JSX
    <Outlet />        // content from a matched child route
  </div>
);

// Usage: parent passes a header via children; the URL fills Outlet
<Dashboard>
  <h1>Title</h1>
</Dashboard>
```

---

17. TypeScript Helpers — Derive Types from Data

typeof — derive a type from a value

```
const data = [{ name: "A", price: 10 }];
```

```
type Item = (typeof data)[0];
// Item = { name: string; price: number }
```

data becomes the source of truth, no duplicate interface. good for mock data, config arrays, etc.

Non-null assertion (!) — "trust me, this exists"

```
const { id } = useParams();   // id: string | undefined
fetchItem(id!);              // !  asserts: not null/undefined
```

> ! silences the compiler — no runtime check. only when the value is actually guaranteed (e.g. route param). otherwise: if (id) { ... }.

Optional chaining (?.) and nullish coalescing (??)

```
const name = response?.data?.[0]?.name;     // undefined if any link is
null/undefined
const count = response?.total ?? 0;         // 0 only if total is null or
undefined
```

---

18. Spread Operator — Quick Reference

... copies items/properties into a new array or object. needed for immutable state updates.

```
// Append to an array
const updated = [...old, newItem];

// Prepend
const updated = [newItem, ...old];

// Merge two arrays
const combined = [...arrA, ...arrB];

// Copy an object, override one field
const updated = { ...user, name: "new name" };

// Spread function args
Math.max(...numbers);
```

> spread is shallow — nested refs are shared. deep clone: structuredClone(value) or copy each level.

---

19. Outlet Context — Pass Data to Nested Routes

nested routes are rendered by the router, not parent JSX — so no props. Outlet context shares a value down to nested children, no drilling.

Parent — share a value via Outlet

```
import { Outlet } from "react-router-dom";

const Parent = ({ value }: { value: string }) => (
  <div>
    <Outlet context={value} />   // share with all nested routes
  </div>
);
```

Child — read the value with useOutletContext

```
import { useOutletContext } from "react-router-dom";

const Child = () => {
  const value = useOutletContext<string>();   // type the value here
  return <div>{value}</div>;
};
```

Passing multiple values

Outlet context only takes one value. To share several, wrap them in an object.

```
// Parent
<Outlet context={{ id, user, refresh }} />

// Child — destructure with a typed shape
type Ctx = { id: string; user: User; refresh: () =&gt; void };
const { id, user, refresh } = useOutletContext<Ctx>();
```

> Outlet context only travels one level — Outlet → direct children. for app-wide state, use React Context (createContext + useContext) or a state library.

---

20. Regex .replace() with a Callback

String.replace's second arg can be a function. called per match, gets the capture groups, returns the replacement. handy when the replacement depends on what was matched.

Static replace vs callback replace

```
// Static — replacement string is fixed
"hello world".replace(/world/, "there");       // "hello there"

// Callback — replacement is computed from each match
"hello world".replace(/(\w+)/g, (match) => match.toUpperCase());
// "HELLO WORLD"
```

Reading captured groups in the callback

The callback receives: (fullMatch, group1, group2, …, offset, fullString). Use the captured groups to build the replacement.

```
// Convert dashes inside ID-like tokens to nothing
const url = "/files/0001234-56-789012.txt";
```

```
const clean = url.replace(
  /\/([\d-]+)\.txt$/,                  // match "/0001234-56-789012.txt"
  (_, id) => \`/\${id.replace(/-/g, "")}.htm\`
);
// "/files/000123456789012.htm"
```

> first arg = full match (use _ if not needed). then capture groups left-to-right. /g flag replaces every match, not just the first.

Common uses

URL rewriting, slug generation, template substitution ({{name}} → value), conditional formatting.

---

21. Wrapped API Responses

APIs nest data differently — bare array, wrapped in results/data/items, or nested two levels deep. on swap, both the axios.get<T> generic and the unwrap path have to match.

Three common response shapes

```
// 1. Bare array
// API returns: [ {..., {...} ]}
const res = await axios.get<Item[]>(url);
const items = res.data;

// 2. Wrapped in a key ("results", "data", "items", etc.)
// API returns: { results: [ {..., {...} ] }}
const res = await axios.get<{ results: Item[] }>(url);
const items = res.data.results;

// 3. Nested with metadata
// API returns: { data: { items: [...], total: 42, page: 1  }}
type Response = { data: { items: Item[]; total: number; page: number } };
const res = await axios.get<Response>(url);
const items = res.data.data.items;
```

> two things have to match: the axios.get<T> generic AND the runtime unwrap path (res.data vs res.data.results vs res.data.data.items). TS won't flag a mismatch between the two.

Defensive reading

optional chaining handles missing wrappers (e.g. error responses). pair with a fallback so consumers always get something usable.

```
const items = res?.data?.results ?? [];      // always an array
const total = res?.data?.meta?.total ?? 0;   // always a number
```

First time inspecting an unfamiliar API

```
const res = await axios.get(url);
console.log(res.data);     // inspect the actual shape before typing it
```

> before typing it, hit the endpoint once with no generic and log the response. write the type to match what's actually there.

Quick Reference

When you want to...

Goal	Method	Quick example
Store a value	useState	`const [x, setX] = useState('')`
Add to array	spread + setState	`[...old, newItem]`
Remove from array	.filter()	`arr.filter(v => v !== target)`
Prevent duplicates	.find() + early return	`if (arr.find(...)) return`
Check item exists	.find() / .includes()	`arr.includes('AAPL')`
Render list	.map()	`arr.map(item => <Card />)`
Show/hide element	&& operator	`{error && <p>{error}</p>}`
Show A or B	ternary ? :	`{x > 0 ? <List /> : <Empty />}`
Stop form page reload	e.preventDefault()	`Always first in form handler`
Read form input value	e.target[0].value	`Index = position in the form`
Call an API	async / await	`const res = await fetchData()`
Handle API error	typeof check	`typeof result === 'string'`
Define props shape	interface Props {}	`id: string; onSubmit: () => void`
Run code on mount	useEffect + []	`useEffect(() => { ... }, [])`
Re-run when value changes	useEffect + [dep]	`useEffect(() => { ... }, [id])`
Cleanup an effect	return fn from effect	`return () => clearInterval(id)`
Define routes	createBrowserRouter	`{ path: '/', element: <App /> }`
Render child route	<Outlet />	`<Outlet />`
In-app navigation link	<Link to=...>	`<Link to='/about'>About</Link>`
Read URL parameter	useParams()	`const { id } = useParams()`
Append to array	spread	`[...old, newItem]`
Override object field	spread	`{ ...obj, name: 'x' }`
Derive type from value	typeof	`type T = (typeof data)[0]`
Assert non-null	! operator	`fetchItem(id!)`
Safe property access	?. chain	`user?.address?.city`
Default for null/undef	?? operator	`count ?? 0`
Pass data to nested routes	<Outlet context={x} />	`<Outlet context={value} />`
Read data in nested route	useOutletContext<T>()	`const v = useOutletContext<string>()`
Replace with computed value	regex + callback	`s.replace(/x/, (m) => f(m))`

Capture groups in replace	(_, group1) =>	`url.replace(/\/(\d+)\.txt/, (_, id) => ...)`
Type a wrapped API response	<{ key: T[] }>	`axios.get<{ results: Item[] }>(url)`
Read wrapped API response	res.data.<key>	`res.data.results`
Safe unwrap with fallback	?. + ??	`res?.data?.results ?? []`

Patterns picked up while building — keep adding as I go.